

T1 Testare unitară în Python - Chiose Mihnea Stefan (343), Armeanca Elis-Daniel (342)

1. 1) CONFIGURAȚIA SISTEMULUI
 - a. STRUCTURA PROIECTULUI:
2. 2) CLASA DE BAZĂ RentalBookingProcessor
 - a. Sectiunea de validari initiale
 - b. Sectiunea de parcurgere si filtrare initiala a masinilor
 - c. Sectiunea de reguli de business si calcul pret
 - d. Sectiunea de selectie finala a masinilor si return
3. 3) PARTIȚIONARE IN CLASE DE ECHIVALENȚĂ
 - a. BUCATI DE COD VIZATE
4. 4) ANALIZA VALORILOR DE FRONTIERĂ
 - a. BUCATI DE COD VIZATE
5. 5) TESTARE STRUCTURALĂ
 - a. STATEMENT COVERAGE
 - b. Bucati de cod vizate
 - c. DECISION COVERAGE
 - d. Bucati de cod vizate
 - e. CONDITION COVERAGE
 - f. Bucati de cod vizate direct
 - g. INDEPENDENT CIRCUITS
 - h. Bucati de cod vizate
 - i. a) trasee de iesire imediata
 - ii. b) trasee de filtrare in bucla
 - iii. c) trasee cu restrictie si buget
 - iv. d) trasee cu acceptare finala
 - v. e) trasee de final

6. 6) MUTAȚII

- a. RAPORTUL INITIAL:
- b. RAPORTUL DE DUȚA TESTELE ADITIONALE:
- c. Teste suplimentare pentru eliminarea mutantilor
- d. Zona de cod afectata de mutatii
 - i. Exemplu de test aditional:

1) CONFIGURAȚIA SISTEMULUI

Testele au fost realizate pe un sistem personal cu următoarele caracteristici:

- Procesor: AMD
- Memorie RAM: 8 GB
- Spațiu de stocare: SSD
- Sistem de operare: Windows 10

Pentru dezvoltare și testare au fost utilizate următoarele tehnologii:

- Limbaj de programare: Python 3.13.2
- Framework de testare: unittest (modul standard din Python)
- IDE: PyCharm
- Sistem de versionare: Git + GitHub
- Sistem de operare: Windows
- Mutatii: *cosmic-ray*

Pentru rularea testelor s-a utilizat comanda:

```
python -m unittest
```

STRUCTURA PROIECTULUI:

tss/

src/*clasa de bază*

tests/*toate testele*

2) CLASA DE BAZĂ *RentalBookingProcessor*

Clasa **RentalBookingProcessor** are rolul de a procesa cererile de inchiriere auto pe baza datelor clientului și a listei de mașini disponibile. Metoda `process_booking(...)` validează mai întâi datele de intrare, apoi analizează fiecare mașină și aplică un set de reguli de business pentru a decide dacă aceasta poate fi oferită clientului.

Regulile urmăresc eligibilitatea clientului și a mașinii din mai multe puncte de vedere:

- vârsta minimă,
- existența permisului de conducere,
- experiența la volan,

- perioada de inchiriere,
- bugetul disponibil,
- numarul minim de locuri,
- disponibilitatea masinii,
- categoria acesteia,
- restrictiile speciale pentru anumite situatii.

Metoda poate returna:

- un mesaj de eroare, daca datele de intrare sunt invalide
- mesajul "No cars available for this client", daca nu exista masini eligibile
- o lista de masini eligibile, daca exista optiuni care respecta toate conditiile

Reguli de business:

- clientul trebuie sa aiba **cel putin 18 ani**
- clientul trebuie sa detina **permis valid**
- **experienta** de condus **nu poate fi negativa**
- **perioada de inchiriere** trebuie sa fie **pozitiva**
- **bugetul** trebuie sa fie **pozitiv**
- **numarul minim de locuri** trebuie sa fie **pozitiv**
- **masina trebuie sa contina toate campurile necesare**
- categoria masinii trebuie sa fie valida: **economy, standard, premium**
- masina trebuie sa fie **disponibila**
- masina trebuie sa aiba **suficiente locuri**
- **pentru clientii sub 21 ani sau cu experienta sub 2 ani, masinile cu peste 120 CP sunt respinse**
- pentru clientii **sub 21 ani** se aplica **taxa suplimentara**
- **masinile premium pot fi inchiriate doar de clienti cu varsta de cel putin 25 ani si experienta de cel putin 3 ani**
- **masinile economy si standard pot fi inchiriate mai usor, daca clientul are cel putin 18 ani si minimum 1 an experienta**

Sectiunea de validari initiale

Aceasta sectiune verifica daca datele de intrare ale clientului sunt valide.

```
if client_age < 18:
    return "Client is underage"
```

```
if not has_license:
    return "Client has no valid license"
```

```
if driving_experience_years < 0:  
    return "Invalid driving experience"
```

```
if rental_days <= 0:  
    return "Invalid rental period"
```

```
if budget <= 0:  
    return "Invalid budget"
```

```
if min_seats_needed <= 0:  
    return "Invalid seat requirement"
```

Secțiunea de parcurgere și filtrare inițială a mașinilor

Această secțiune parcurge lista de mașini și elimină mașinile care nu îndeplinesc condițiile minime.

```
valid_categories = ["economy", "standard", "premium"]  
available_options = []
```

```
for car in cars:  
    if (  
        "model" not in car  
        or "category" not in car  
        or "price_per_day" not in car  
        or "available" not in car  
        or "horsepower" not in car  
        or "seats" not in car  
    ):  
        continue
```

```
if car["category"] not in valid_categories:  
    continue
```

```
if not car["available"]:  
    continue
```

```
if car["seats"] < min_seats_needed:  
    continue
```

Sețiunea de reguli de business și calcul pret

Aceasta secțiune aplică regulile de închiriere și calculează prețul total.

```
if client_age < 21 or driving_experience_years < 2:
    if car["horsepower"] > 120:
        continue

base_price = car["price_per_day"] * rental_days
extra_fee = 0

if client_age < 21:
    extra_fee = 20 * rental_days

total_price = base_price + extra_fee

if total_price > budget:
    continue
```

Sețiunea de selecție finală a mașinilor și return

Această secțiune decide dacă mașina se adaugă în rezultat și ce returnează metoda la final.

```
if car["category"] == "premium":
    if client_age >= 25 and driving_experience_years >= 3:
        available_options.append(
            {
                "model": car["model"],
                "category": car["category"],
                "total_price": total_price,
                "horsepower": car["horsepower"],
                "seats": car["seats"]
            }
        )
    else:
        if client_age >= 18 and driving_experience_years >= 1:
            available_options.append(
                {
                    "model": car["model"],
                    "category": car["category"],
                    "total_price": total_price,
                    "horsepower": car["horsepower"],
```

```

"seats": car["seats"]
}
)

if len(available_options) == 0:
    return "No cars available for this client"

return available_options

```

3) PARTIȚIONARE IN CLASE DE ECHIVALENȚĂ

Pentru testarea functională a metodei `process_booking(...)` a fost folosită tehnica partiționării în clase de echivalență. **Aceasta presupune împartirea domeniului de intrare în categorii de valori care sunt tratate în mod similar de specificația metodei. Din fiecare clasă relevantă se alege un reprezentant, obținându-se astfel un set redus, dar semnificativ, de teste.**

În cazul metodei analizate, au fost identificate atât clase invalide, cât și clase valide. **Clasele invalide** corespund situațiilor în care **datele clientului nu respectă condițiile minime de procesare, de exemplu:** *client minor, lipsa permisului, experiența negativă, perioada de închiriere invalidă, buget invalid sau cerință invalidă privind numărul de locuri.* **Clasele valide** includ cazuri în care **clientul poate închiria mașini economy sau standard, precum și cazul în care clientul îndeplinește condițiile pentru mașini premium.**

Testele implementate în fișierul `test_equivalence_partitioning.py` verifică reprezentanții ai acestor clase și confirmă faptul că metoda produce rezultatul așteptat pentru fiecare categorie importantă de intrări.

Avem 9 teste:

1. `test_underage_client`
2. `test_client_without_license`
3. `test_negative_driving_experience`
4. `test_invalid_rental_period`
5. `test_invalid_budget`
6. `test_invalid_seat_requirement`
7. `test_no_cars_available_empty_list`
8. `test_valid_economy_or_standard_option_exists`
9. `test_premium_allowed_for_eligible_client`

BUCĂȚI DE COD VIZATE

- VALIDĂRILE INITIALE

- Cazul in care nu exista masini eligibile:

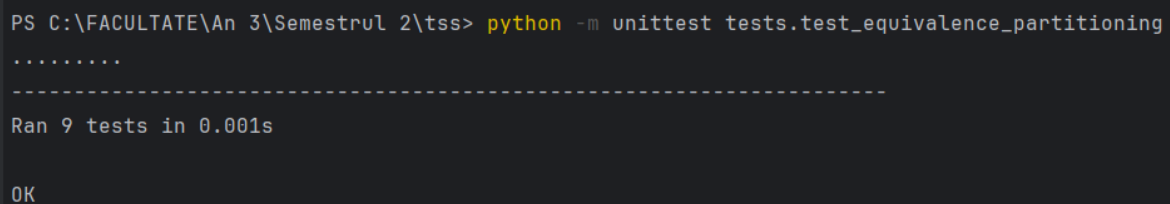
```
if len(available_options) == 0:
    return "No cars available for this client"
```

- Cazul pentru economy/standard

```
if car["category"] == "premium":
    ...
else:
    if client_age >= 18 and driving_experience_years >= 1:
        available_options.append(...)
```

- Cazul valid pentru premium

```
if car["category"] == "premium":
    if client_age >= 25 and driving_experience_years >= 3:
        available_options.append(...)
```



```
PS C:\FACULTATE\An 3\Semestrul 2\tss> python -m unittest tests.test_equivalence_partitioning
.....
-----
Ran 9 tests in 0.001s

OK
```

4) ANALIZA VALORILOR DE FRONTIERĂ

Pe langa clasele de echivalenta, a fost folosita si tehnica analizei valorilor de frontiera. Aceasta se concentreaza pe valorile aflate la limita dintre clase, deoarece aceste zone sunt predispuse la erori de implementare.

Pentru metoda `process_booking(...)` au fost identificate urmatoarele praguri importante:

- **varsta clientului:** 18, 21 si 25 ani
- **experienta la volan:** 0 si 1 ani
- **perioada de inchiriere:** pragul dintre 0 si 1 zi
- **bugetul disponibil:** sub pretul total, exact egal cu pretul total si peste pretul total
- **puterea masinii:** pragul de 120 CP pentru clientii restrictionati

Testele implementate in fisierul `test_boundary_values.py` verifica aceste valori si confirma faptul ca metoda trateaza corect trecerile dintre cazurile valide si cele invalide, precum si dintre cazurile eligibile si cele neeligibile.

Avem 8 teste:

1. test_client_age_boundary_17_18
2. test_client_age_boundary_20_21_extra_fee
3. test_client_age_boundary_24_25_for_premium
4. test_driving_experience_boundary_minus1_0
5. test_driving_experience_boundary_0_1
6. test_rental_days_boundary_0_1
7. test_budget_boundary_under_exact_over_for_young_driver
8. test_horsepower_boundary_120_121_for_restricted_driver

BUCATI DE COD VIZATE

- Frontiera de varsta 17 / 18

```
if client_age < 18:  
    return "Client is underage"
```

- Frontiera de varsta 20 / 21

```
if client_age < 21:  
    extra_fee = 20 * rental_days
```

- Frontiera de varsta 24 / 25

```
if car["category"] == "premium":  
    if client_age >= 25 and driving_experience_years >= 3:  
        available_options.append(...)
```

- Frontiera experienta -1 / 0

```
if driving_experience_years < 0:  
    return "Invalid driving experience"
```

- Frontiera experienta 0 / 1

```
else:  
    if client_age >= 18 and driving_experience_years >= 1:  
        available_options.append(...)
```

- Frontiera rental days 0 / 1

```
if rental_days <= 0:  
    return "Invalid rental period"
```

- Frontiera de buget


```
base_price = car["price_per_day"] * rental_days
extra_fee = 0
```

```
if client_age < 21:
    extra_fee = 20 * rental_days
```

```
total_price = base_price + extra_fee
```

```
if total_price > budget:
    continue
```

- Frontiera horsepower 120 / 121

```
if client_age < 21 or driving_experience_years < 2:
    if car["horsepower"] > 120:
        continue
```

```
PS C:\FACULTATE\An 3\Semestrul 2\tss> python -m unittest tests.test_boundary_values
.....
-----
Ran 10 tests in 0.002s

OK
```

5) TESTARE STRUCTURALĂ

Testarea structurala genereaza cazuri de test pe baza implementarii, urmarind acoperirea unor elemente din graful de control: instructiuni, decizii, conditii si cai/circuite.

Principiul fisier este *test_structural_coverage.py* contine:

- TestStatementCoverage
- TestDecisionCoverage
- TestConditionCoverage
- TestIndependentCircuits

STATEMENT COVERAGE

Pentru acoperirea la nivel de instructiune au fost alese teste care executa atat trasee de iesire imediata din validari, cat si trasee complete prin bucla de procesare a masinilor. Obiectivul a fost executarea principalelor blocuri de cod cel putin o data: validari initiale, filtre aplicate fiecarei masini, calculul pretului total, adaugarea unei optiuni eligibile si ramura finala in care nu exista masini disponibile.

1. test_statement_invalid_input_path

Verifica iesirea imediata din metoda pentru client minor.

2. test_statement_main_processing_path

Verifica fluxul principal al metodei (parcurgere masini, filtre, calcul pret, adaugare rezultate).

3. test_statement_no_available_options

Verifica cazul in care nu exista masini eligibile.

Bucati de cod vizate

```
if client_age < 18:  
    return "Client is underage"
```

```
if not has_license:  
    return "Client has no valid license"
```

```
if driving_experience_years < 0:  
    return "Invalid driving experience"
```

```
if rental_days <= 0:  
    return "Invalid rental period"
```

```
if budget <= 0:  
    return "Invalid budget"
```

```
if min_seats_needed <= 0:  
    return "Invalid seat requirement"
```

```
for car in cars:  
    if (  
        "model" not in car  
        or "category" not in car  
        or "price_per_day" not in car  
        or "available" not in car  
        or "horsepower" not in car  
        or "seats" not in car  
    ):  
        continue
```

```
base_price = car["price_per_day"] * rental_days  
extra_fee = 0
```

```

if client_age < 21:
    extra_fee = 20 * rental_days

total_price = base_price + extra_fee

if car["category"] == "premium":
    if client_age >= 25 and driving_experience_years >= 3:
        available_options.append(
            {
                "model": car["model"],
                "category": car["category"],
                "total_price": total_price,
                "horsepower": car["horsepower"],
                "seats": car["seats"]
            }
        )
    else:
        if client_age >= 18 and driving_experience_years >= 1:
            available_options.append(
                {
                    "model": car["model"],
                    "category": car["category"],
                    "total_price": total_price,
                    "horsepower": car["horsepower"],
                    "seats": car["seats"]
                }
            )

if len(available_options) == 0:
    return "No cars available for this client"

return available_options

```

DECISION COVERAGE

Pentru acoperirea la nivel de decizie au fost proiectate teste astfel incat deciziile relevante sa fie evaluate atat pe ramura True, cat si pe ramura False. Au fost vizate atat deciziile de validare initiala, cat si deciziile din interiorul buclei de filtrare a masinilor, precum verificarea

disponibilitatii, categoriei, numarului de locuri, restrictiei de putere, incadrarii in buget si eligibilitatii pentru categoria premium.

1. test_decision_client_without_license

Verifica decizia pentru client fara permis.

2. test_decision_invalid_experience

Verifica decizia pentru experienta negativa.

3. test_decision_filters_inside_loop

Verifica deciziile din bucla: campuri lipsa, categorie invalida, indisponibilitate, locuri insuficiente, putere prea mare.

4. test_decision_budget_exceeded

Verifica decizia cand pretul depaseste bugetul.

5. test_decision_premium_rejected

Verifica cazul in care masina premium este respinsa din cauza conditiilor neindeplinite.

6. test_decision_standard_rejected_low_experience

Verifica respingerea unei masini standard din cauza experientei insuficiente.

Bucati de cod vizate

```
if client_age < 18:  
    return "Client is underage"
```

```
if not has_license:  
    return "Client has no valid license"
```

```
if driving_experience_years < 0:  
    return "Invalid driving experience"
```

```
if rental_days <= 0:  
    return "Invalid rental period"
```

```
if budget <= 0:  
    return "Invalid budget"
```

```
if min_seats_needed <= 0:  
    return "Invalid seat requirement"
```

```
if (  
    "model" not in car  
    or "category" not in car
```

```

or "price_per_day" not in car
or "available" not in car
or "horsepower" not in car
or "seats" not in car
):
continue

if car["category"] not in valid_categories:
continue

if not car["available"]:
continue

if car["seats"] < min_seats_needed:
continue


if client_age < 21 or driving_experience_years < 2:
if car["horsepower"] > 120:
continue


if client_age < 21:
extra_fee = 20 * rental_days


if total_price > budget:
continue


if car["category"] == "premium":
if client_age >= 25 and driving_experience_years >= 3:
available_options.append(...)
else:
if client_age >= 18 and driving_experience_years >= 1:
available_options.append(...)

```

```
return "No cars available for this client"
```

CONDITION COVERAGE

Pentru acoperirea la nivel de conditie s-a insistat asupra expresiilor booleene compuse din metoda. In special, s-au testat separat valorile componentelor individuale ale conditiilor *client_age < 21 or driving_experience_years < 2*, *client_age >= 25 and driving_experience_years >= 3* si *client_age >= 18 and driving_experience_years >= 1*, astfel incat fiecare conditie individuala sa ia atat valoarea True, cat si False.

- 1. test_condition_or_age_true_experience_false**

Prima conditie True, a doua False pentru expresia cu OR.

- 2. test_condition_or_age_false_experience_true**

Prima conditie False, a doua True pentru expresia cu OR.

- 3. test_condition_or_both_false**

Ambele conditii False, masina este acceptata.

- 4. test_condition_premium_and_true_false**

Pentru conditia AND, prima True, a doua False (premium respins).

- 5. test_condition_standard_and_true_false**

Pentru conditia AND, prima True, a doua False (standard respins).

Bucati de cod vizate direct

```
if client_age < 21 or driving_experience_years < 2:  
    if car["horsepower"] > 120:  
        continue
```

```
if car["category"] == "premium":  
    if client_age >= 25 and driving_experience_years >= 3:  
        available_options.append(  
            {  
                "model": car["model"],  
                "category": car["category"],  
                "total_price": total_price,  
                "horsepower": car["horsepower"],  
                "seats": car["seats"]  
            }  
        )
```

```

else:
    if client_age >= 18 and driving_experience_years >= 1:
        available_options.append(
            {
                "model": car["model"],
                "category": car["category"],
                "total_price": total_price,
                "horsepower": car["horsepower"],
                "seats": car["seats"]
            }
        )

```

INDEPENDENT CIRCUITS

Pentru testarea circuitelor independente s-au identificat trasee de baza prin metoda, corespunzatoare unor cai distincte de executie: iesiri imediate in etapa de validare, filtre succesive in bucla de prelucrare a masinilor, trasee care conduc la respingerea unei masini si trasee care conduc la adaugarea unei optiuni valide. Aceste trasee au fost folosite ca baza pentru construirea unei suite de teste structurale reprezentative.

- 1. test_circuit_1_underage_exit**
Iesire imediata pentru client minor.
- 2. test_circuit_2_no_license_exit**
Iesire pentru client fara permis.
- 3. test_circuit_3_missing_field_continue**
Masina eliminata din cauza campurilor lipsa.
- 4. test_circuit_4_invalid_category_continue**
Masina eliminata pentru categorie invalida.
- 5. test_circuit_5_unavailable_continue**
Masina eliminata pentru indisponibilitate.
- 6. test_circuit_6_not_enough_seats_continue**
Masina eliminata pentru locuri insuficiente.
- 7. test_circuit_7_restricted_driver_high_horsepower_continue**
Masina eliminata pentru putere prea mare (client restrictionat).
- 8. test_circuit_8_standard_added**
Masina economy/standard este acceptata.
- 9. test_circuit_9_premium_added**
Masina premium este acceptata.

10. test_circuit_10_budget_continue_final_no_options

Masina eliminata din cauza bugetului, rezultat final fara optiuni.

Bucati de cod vizate

a) trasee de iesire imediata

```
if client_age < 18:
    return "Client is underage"

if not has_license:
    return "Client has no valid license"
```

b) trasee de filtrare in bucla

```
for car in cars:
    if (
        "model" not in car
        or "category" not in car
        or "price_per_day" not in car
        or "available" not in car
        or "horsepower" not in car
        or "seats" not in car
    ):
        continue

    if car["category"] not in valid_categories:
        continue

    if not car["available"]:
        continue

    if car["seats"] < min_seats_needed:
        continue
```

c) trasee cu restrictie si buget

```
if client_age < 21 or driving_experience_years < 2:
    if car["horsepower"] > 120:
        continue

base_price = car["price_per_day"] * rental_days
extra_fee = 0
```



```

if client_age < 21:
    extra_fee = 20 * rental_days

total_price = base_price + extra_fee

if total_price > budget:
    continue

```

d) trasee cu acceptare finala

```

if car["category"] == "premium":
    if client_age >= 25 and driving_experience_years >= 3:
        available_options.append(
            {
                "model": car["model"],
                "category": car["category"],
                "total_price": total_price,
                "horsepower": car["horsepower"],
                "seats": car["seats"]
            }
        )
    else:
        if client_age >= 18 and driving_experience_years >= 1:
            available_options.append(
                {
                    "model": car["model"],
                    "category": car["category"],
                    "total_price": total_price,
                    "horsepower": car["horsepower"],
                    "seats": car["seats"]
                }
            )

```

e) trasee de final

```

if len(available_options) == 0:
    return "No cars available for this client"

return available_options

```

SCREENSHOT STRUCTURAL COVERAGE:

```
PS C:\FACULTATE\An 3\Semestrul 2\tss> python -m unittest tests.test_structural_coverage
.....
-----
Ran 24 tests in 0.003s

OK
```

6) MUTAȚII

Am generat raport de mutatii cu biblioteca cosmic-ray.

```
pip install cosmic-ray
```

COSMIC-RAY.TOML:

```
[cosmic-ray]
module-path = "src/rental_booking_processor.py"
test-command = "python -m unittest discover -s tests"
timeout = 10
```

```
[cosmic-ray.distributor]
name = "local"
```

```
cosmic-ray init cosmic-ray.toml session.sqlite
```

```
cosmic-ray baseline cosmic-ray.toml
cosmic-ray exec cosmic-ray.toml session.sqlite
cr-report session.sqlite
```

RAPORTUL INITIAL:

- total jobs: **191**
- surviving mutants: **24**
- rata bruta de mutanti omorati: **167 / 191 = 87.43%**

```
worker outcome: normal, test outcome: killed
total jobs: 191
complete: 191 (100.00%)
surviving mutants: 24 (12.57%)
```

În urma rularii initiale a mutation testing, au fost identificati 24 de mutanti supravietuitori, ceea ce indica faptul ca anumite scenarii nu erau acoperite suficient de bine de testele

existente.

Au fost analizati mutanti relevanti, in special cei de tip ReplaceContinueWithBreak, care modifica logica de parcurgere a listei de masini.

Pentru acestia, au fost dezvoltate teste suplimentare care verifica comportamentul in cazul unor masini invalide urmate de masini valide, pentru a observa daca iteratia continua corect.

Dupa adaugarea testelor suplimentare si rerularea analizei, numarul mutantilor supravietuitori a scazut la 20, ceea ce confirma imbunatatirea acoperirii testelor.

Acest lucru demonstreaza ca testele initiale nu acopereau complet toate cazurile logice, iar mutation testing a ajutat la identificarea acestor lacune.

RAPORTUL DE DUPA TESTELE ADITIONALE:

```
worker outcome: normal, test outcome: killed
total jobs: 191
complete: 191 (100.00%)
surviving mutants: 20 (10.47%)
```

Teste suplimentare pentru eliminarea mutantilor

Pentru mutantii supravietuitori de tip ReplaceContinueWithBreak, a fost identificata problema ca testele initiale nu verificau corect parcurgerea completa a listei de masini.

Mutatia ReplaceContinueWithBreak inlocuieste instructiunea "continue" cu "break", ceea ce duce la oprirea prematura a iteratiei prin lista de masini.

Pentru a detecta acest comportament, au fost adaugate teste suplimentare care contin:

- masini invalide urmate de masini valide
- verificarea faptului ca masina valida este totusi luata in considerare

Exemplu de scenariu:

- prima masina este invalida (nu indeplineste conditiile)
- a doua masina este valida
- comportamentul corect: a doua masina trebuie returnata
- comportament mutant: iteratia se opreste si nu mai este evaluata masina valida

Zona de cod afectata de mutatii

Mutatiile analizate afecteaza instructiunile de control al fluxului din bucla de parcurgere a masinilor:

```
for car in cars:
...
if car["category"] not in valid_categories:
```

```
continue
```

```
if not car["available"]:  
    continue
```

```
if car["seats"] < min_seats_needed:  
    continue
```

Inlocuirea instructiunii "continue" cu "break" modifica semnificativ comportamentul programului, deoarece bucla se opreste prematur si nu mai sunt evaluate toate masinile disponibile.

Acest tip de eroare este critic deoarece poate duce la eliminarea unor optiuni valide pentru client.

Exemplu de test aditional:

```
# test care verifica daca bucla continua corect iteratia (continue) si nu se  
opreste (break)
```

```
def test_continue_not_break_behavior(self):
```

```
# lista de masini:
```

```
# prima masina este invalida (categorie gresita)
```

```
# a doua masina este valida
```

```
cars = [  
    {
```

```
        "model": "InvalidCar",  
        "category": "invalid", # categorie invalida -> trebuie sarita
```

```
        "price_per_day": 50,  
        "available": True,
```

```
        "horsepower": 100,  
        "seats": 4
```

```
    },
```

```
    {
```

```
        "model": "ValidCar",  
        "category": "economy", # masina valida -> trebuie luata in calcul  
        "price_per_day": 50,
```

```
        "available": True,  
        "horsepower": 100,
```

```
        "seats": 4
```

```
]
```

```
# apelam metoda cu date valide pentru client
```

```
result = self.processor.process_booking(
```

```
cars,
```

```
client_age=25,
```

```
has_license=True,
```

```
driving_experience_years=5,
```

```
rental_days=2,
```

```
budget=500,
```

```
min_seats_needed=4
```

```
)
```

```
# daca mutatia "continue -> break" exista:
```

```
# bucla se opreste la prima masina invalida
```

```
# => nu mai ajunge la masina valida
```

```
# => rezultatul ar fi "No cars available..."
```

```
# testul verifica ca acest comportament NU apare
```

```
self.assertNotEqual(result, "No cars available for this client")
```